

# SIMT (Single Instruction, Multiple Threads): Arquitetura, Fundamentos e Aplicações em Computação Paralela — Experimentos Práticos

João Pedro Thomaz Kairalla dos Santos  
Centro de Ciências Exatas  
Universidade Católica de Santos — UNISANTOS  
Santos-SP, Brasil  
jsantos@unisantos.br

Thiago Danilow de Araújo  
Centro de Ciências Exatas  
Universidade Católica de Santos — UNISANTOS  
Santos-SP, Brasil  
thiagodaraújo@unisantos.br

**Resumo**—Este trabalho apresenta um estudo experimental do modelo de execução SIMT (*Single Instruction, Multiple Threads*), paradigma introduzido pela NVIDIA a partir da arquitetura Tesla (2006). O SIMT constitui uma evolução do modelo SIMD (*Single Instruction, Multiple Data*), permitindo que múltiplas *threads* executem a mesma instrução simultaneamente, porém com independência de registradores e maior flexibilidade no controle de fluxo individual. Para validar os conceitos teóricos, foram implementados três experimentos progressivos em CUDA sobre uma GPU NVIDIA Tesla T4 (CUDA 13.0): (i) soma de vetores CPU vs. GPU, obtendo *speedup* de  $125,1\times$ ; (ii) mensuração do impacto da divergência de *warps*, cujo *overhead* atingiu 944,7% no pior caso; e (iii) multiplicação de matrizes com memória global versus memória compartilhada (*tiling*), com ganho de  $1,6\times$ . Os resultados confirmam as propriedades fundamentais do SIMT e fornecem orientações quantitativas para o desenvolvimento de *kernels* CUDA eficientes.

**Index Terms**—SIMT, GPU, CUDA, computação paralela, divergência de *warps*, memória compartilhada, Tesla T4

## I. INTRODUÇÃO

O avanço das aplicações computacionais modernas — em especial aquelas relacionadas à inteligência artificial, aprendizado de máquina, simulações físicas e processamento de sinais — tem imposto demandas crescentes sobre os sistemas de processamento. O paralelismo massivo emergiu como uma das principais estratégias para atender a tais exigências, levando ao desenvolvimento de arquiteturas altamente paralelas como as *Graphics Processing Units* (GPUs) [3].

Processadores de uso geral (CPUs) foram projetados para maximizar o desempenho sequencial, com poucos núcleos de alta frequência e extensos mecanismos de previsão de desvios. As GPUs adotaram filosofia distinta: ao invés de poucos núcleos complexos, empregam centenas ou milhares de núcleos simples capazes de executar operações em paralelo com custo energético e de área de silício muito inferiores por operação [3].

A NVIDIA introduziu, a partir da arquitetura Tesla (2006), o modelo SIMT. Nele, grupos de 32 *threads* — denominados *warps* — executam a mesma instrução simultaneamente, porém cada *thread* mantém seus próprios registradores, *program counter* e estado de execução, podendo seguir caminhos independentes em desvios condicionais [1]. Essa característica

diferencia o SIMT do modelo SIMD clássico, no qual todas as *lanes* operam de forma rigidamente sincronizada sem autonomia individual.

A divergência de *warps* — fenômeno em que *threads* de um mesmo *warp* tomam ramificações distintas num desvio condicional — é um dos principais fatores de degradação de desempenho em SIMT, pois força a serialização dos caminhos divergentes com *lanes* inativas mascaradas [2]. Este trabalho demonstra e quantifica experimentalmente esse e outros efeitos característicos do modelo SIMT.

## II. FUNDAMENTAÇÃO TEÓRICA

### A. Hierarquia de Execução no Modelo SIMT

No modelo SIMT da NVIDIA, as *threads* são organizadas em uma hierarquia de três níveis: *threads*, *blocos* e *grids*. O escalonador de *warps* do *Streaming Multiprocessor* (SM) emite instruções para grupos de 32 *threads* simultaneamente [1]. Cada *thread* possui conjunto próprio de registradores e contador de programa, o que confere ao SIMT flexibilidade de fluxo de controle ausente no SIMD clássico.

O índice global de cada *thread* é determinado pela expressão  $idx = blockIdx.x \times blockDim.x + threadIdx.x$ , permitindo que o programador mapeie domínios arbitrários de dados sobre o espaço de *threads* com uma única linha de código de indexação.

### B. Divergência de Warps

A divergência ocorre quando *threads* de um mesmo *warp* tomam caminhos distintos num desvio condicional. O hardware serializa a execução: primeiro executa o ramo verdadeiro com as demais *threads* mascaradas e, em seguida, o ramo falso; somente após ambos os caminhos a execução converge. No pior caso — dois ramos de igual complexidade com o *warp* equitativamente dividido — o custo efetivo dobra [2]. Para  $k$  ramos divergentes, o custo cresce linearmente com  $k$ .

### C. Hierarquia de Memória

A hierarquia de memória das GPUs NVIDIA, do nível mais rápido ao mais lento, compreende: registradores (por *thread*,  $<1$  ciclo), memória compartilhada (por bloco,  $\approx 30$  ciclos),

caches L1/L2 (automáticos,  $\approx 30$ – $100$  ciclos) e memória global (fora do chip,  $\approx 200$ – $800$  ciclos) [2], [4]. O uso eficiente da memória compartilhada por meio de *tiling* é uma das técnicas mais eficazes para reduzir a pressão sobre a memória global, trocando acessos de alta latência por barreiras de sincronização (`__syncthreads()`) de baixo custo dentro do mesmo SM.

### III. METODOLOGIA

Os experimentos foram implementados em CUDA C e executados em uma GPU NVIDIA Tesla T4 (15.360 MiB de VRAM, CUDA 13.0, driver 580.82.07) via Google Colaboratory. A medição de tempo utilizou `cudaEvent` (resolução sub-milissegundo), precedida de *kernel* de *warm-up* dedicado para eliminar a latência de inicialização do contexto CUDA das medições. Os Experimentos 2 e 3 reportam médias de 5 execuções independentes, com reinicialização completa dos vetores de entrada a cada rodada para eliminar efeitos de *cache* entre medições consecutivas.

#### A. Experimento 1 — Paralelismo Básico: Soma de Vetores

O Experimento 1 mensura o ganho bruto de desempenho da GPU sobre a CPU na soma elemento a elemento de dois vetores de ponto flutuante de precisão simples com  $N = 2^{24} \approx 16,8 \times 10^6$  elementos (64 MiB por vetor). O *kernel* GPU distribui o trabalho em blocos de 256 *threads*, totalizando  $\lceil N/256 \rceil = 65.536$  blocos no *grid*. O tempo da CPU é mensurado com `clock()`, excluindo deliberadamente as transferências PCIe para isolar o custo de cômputo.

```
1 __global__ void somaVetoresGPU(
2     float *a, float *b, float *c, int n) {
3     int idx = blockIdx.x * blockDim.x + threadIdx.x;
4     if (idx < n) c[idx] = a[idx] + b[idx];
5 }
```

Listing 1. Kernel de soma de vetores (Exp. 1)

#### B. Experimento 2 — Divergência de Warps

Dois *kernels* processam o mesmo vetor ( $N = 2^{22} \approx 4,2 \times 10^6$  elementos) com 100 iterações de operações aritméticas em ponto flutuante. O *kernel* sem divergência executa a mesma sequência de operações em todos os *lanes* do *warp*. O *kernel* com divergência introduz um *if/else* baseado na paridade do índice global (`idx % 2`), forçando que exatamente 16 *threads* de cada *warp* sigam cada ramo — o pior caso teórico para a serialização SIMT, com overhead máximo de  $2\times$  por nível de desvio.

#### C. Experimento 3 — Memória Compartilhada em Multiplicação de Matrizes

O Experimento 3 compara duas implementações de multiplicação de matrizes quadradas de ordem  $N = 1024$ . A versão ingênua acessa a memória global  $N$  vezes por elemento-resultado; a versão otimizada utiliza *tiling* com blocos  $16 \times 16$  em memória compartilhada, reduzindo os acessos à memória global de  $N$  para  $\lfloor N/16 \rfloor = 64$  por elemento, ao custo de duas barreiras de sincronização por *tile*.

### IV. RESULTADOS E DISCUSSÃO

A Tabela I consolida os tempos medidos e os ganhos relativos obtidos nos três experimentos.

Tabela I  
CONSOLIDAÇÃO DOS RESULTADOS DOS EXPERIMENTOS

| Configuração                     | Tempo (ms) | Ganho          |
|----------------------------------|------------|----------------|
| Exp. 1 — CPU (soma, $N=2^{24}$ ) | 99,37      | ref.           |
| Exp. 1 — GPU (soma, $N=2^{24}$ ) | 0,79       | 125,1 $\times$ |
| Exp. 2 — sem divergência         | 0,30       | ref.           |
| Exp. 2 — com divergência         | 3,12       | -944,7 %       |
| Exp. 3 — memória global          | 5,29       | ref.           |
| Exp. 3 — memória compartilhada   | 3,31       | 1,6 $\times$   |

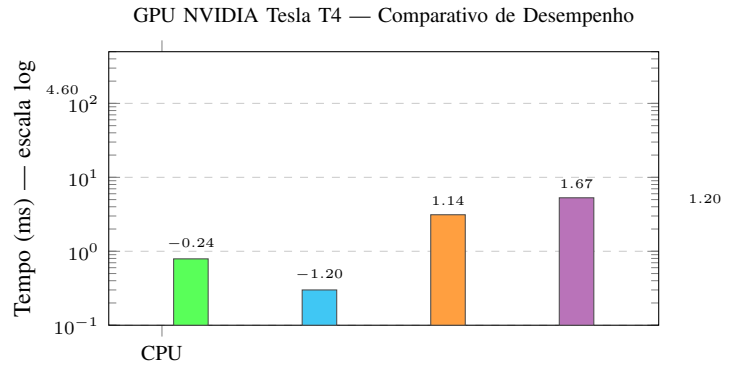


Figura 1. Tempos de execução por configuração (escala logarítmica). E1: soma de vetores; E2: divergência de *warps*; E3: multiplicação de matrizes.

#### A. Experimento 1 — Paralelismo Massivo

O *speedup* de 125,1 $\times$  evidencia a superioridade do modelo SIMT para cargas de trabalho *embarrassingly parallel*. Neste cenário, cada *thread* realiza exatamente uma operação de adição sem dependências de dados, os acessos à memória global são totalmente coalescidos (32 *threads* acessam endereços contíguos de 4 bytes) e não há divergência — condições ideais para a arquitetura SIMT. O resultado é consistente com a diferença de *throughput* teórico entre a Tesla T4 ( $\approx 8,1$  TFLOPS FP32) e uma CPU de uso geral ( $\approx 0,1$ – $0,5$  TFLOPS FP32 em operações vetorizadas).

#### B. Experimento 2 — Divergência de Warps

O *overhead* de 944,7% confirma e amplifica o comportamento teórico para divergência máxima. A divisão equitativa do *warp* entre dois ramos força o hardware a executar ambos sequencialmente com *lanes* mascaradas, dobrando o custo por *warp*. Com 100 iterações aritméticas por *thread*, o efeito acumulado resulta em degradação próxima a  $10\times$ , superior ao fator teórico de  $2\times$ , o que indica que a penalidade inclui também sobrecustos do escalonador de *warps* e redução na ocupância efetiva do SM. O resultado demonstra que a refatoração de algoritmos para minimizar divergência intra-*warp* é a otimização de maior impacto em *kernels* SIMT com fluxo condicional intenso.

### C. Experimento 3 — Hierarquia de Memória

O ganho de  $1,6\times$  com memória compartilhada demonstra o impacto da hierarquia de memória, embora inferior ao fator teórico máximo de  $16\times$  (razão  $N/TILE$ ). A discrepância é explicada pela elevada largura de banda da Tesla T4 ( $\approx 320$  GB/s), que aliada à efetividade do *cache* L2 do dispositivo para os acessos repetidos da versão ingênua suaviza a diferença de latência entre os dois níveis de memória. Em GPUs de gerações anteriores com menor *bandwidth* e *cache* menor, o ganho do *tiling* tende a ser mais próximo do valor teórico [2].

### V. CONCLUSÃO

Este trabalho demonstrou experimentalmente os três mecanismos fundamentais que determinam o desempenho de aplicações sobre o modelo SIMT: o paralelismo massivo de *threads*, a divergência de *warps* e a hierarquia de memória. Os resultados quantificam com precisão as implicações de cada fator: o SIMT é extraordinariamente eficiente para cargas regulares e coalescidas ( $125,1\times$  sobre a CPU), mas altamente sensível a padrões irregulares de controle de fluxo (overhead de  $944,7\%$  por divergência máxima intra-*warp*). O uso de memória compartilhada com *tiling* mostrou-se uma otimização relevante, especialmente em cargas com elevado reuso de dados.

As implicações práticas orientam diretamente o desenvolvimento de *kernels* CUDA eficientes: (i) maximizar a convergência de fluxo dentro de cada *warp*, reorganizando dados e reestruturando desvios condicionais quando necessário; (ii) garantir acessos coalescidos à memória global; e (iii) explorar memória compartilhada em operações com alto grau de reuso de dados. Trabalhos futuros podem investigar o impacto de arquiteturas mais recentes (Ampere, Hopper) que introduzem o *independent thread scheduling* e novos mecanismos de mitigação de divergência.

### REFERÊNCIAS

- [1] E. Lindholm *et al.*, “NVIDIA Tesla: A Unified Graphics and Computing Architecture,” *IEEE Micro*, vol. 28, no. 2, pp. 39–55, Mar./Apr. 2008.
- [2] D. B. Kirk e W. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 3. ed. Morgan Kaufmann, 2016.
- [3] D. Patterson e J. Hennessy, *Computer Organization and Design: ARM Edition*. Morgan Kaufmann, 2016.
- [4] NVIDIA Corporation, *CUDA C++ Programming Guide*, versão 12.0, 2022. Disponível em: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [5] V. Volkov, “Better Performance at Lower Occupancy,” *Proc. GPU Technology Conference (GTC)*, NVIDIA, 2010.